CAMPUSAI

AUTOMATA THEORY AND COMPILER DESIGN (ATCD) – SAMPLE NOTES

1. Introduction

Automata Theory studies abstract machines and the formal languages they recognise. Compiler Design explains how a program written in a high-level language is translated into a target program.

2. Basic Terms in Automata Theory

- **Alphabet (Σ):** A finite, non-empty set of symbols, for example {0, 1}.
- **String:** A finite sequence of symbols from an alphabet. The empty string is written as ε.
- **Language:** A set of strings over an alphabet.
- **Finite Automaton (FA):** A mathematical machine that reads input symbols and changes states.
- **DFA:** A deterministic finite automaton has exactly one next state for each state and input symbol.
- **NFA:** A nondeterministic finite automaton may have more than one possible next state, including ε-transitions in an ε-NFA.
- **Regular Expression:** A notation used to describe regular languages.

3. Grammar and Languages

A formal grammar defines how strings in a language can be formed. A context-free grammar (CFG) consists of variables, terminals, production rules and a start symbol. A parse tree shows the derivation structure of a string.

4. Main Phases of a Compiler

- **Lexical analysis:** Groups characters into tokens such as identifiers, keywords and operators.
- **Syntax analysis:** Checks grammatical structure and builds a syntax tree.
- **Semantic analysis:** Checks meaning, declarations and type rules.
- **Intermediate-code generation:** Produces a machine-independent representation.
- **Code optimization:** Improves code efficiency while preserving its meaning.
- **Code generation:** Produces target or machine code.

5. Example

For the expression total = a + b, lexical analysis identifies names and operators, syntax analysis checks the expression structure, and later compiler phases check meaning and generate target code.

6. Quick Revision Questions

1. Differentiate DFA and NFA.
2. Define alphabet, string and language.
3. What is a regular expression?
4. List the main phases of a compiler.

Note: These are introductory sample notes. Refer to your course syllabus and class materials for complete coverage.